

Pipeline de Machine Learning con Entry Points de Python

Adaptación de un pipeline de scikit-learn para producción

Notebook y paquete auxiliar actualizados para exponer un pipeline de scikit-learn mediante entry points, responder las preguntas de la actividad y documentar la relación con Makefile.

Archivo fuente: Pipeline.ipynb

Sesión: 02/10/2026

Repositorio: <https://github.com/donmatthiuz/MLOPS/tree/actividad9> (<https://github.com/donmatthiuz/MLOPS/tree/actividad9>)

1. Business Understanding

El objetivo del ejercicio es predecir la supervivencia de pasajeros del Titanic para entender qué factores influyeron en las tasas de supervivencia. Desde la perspectiva de minería de datos, este es un problema de **clasificación binaria** donde la variable objetivo es Survived.

El entregable técnico será un pipeline reproducible que integre:

- carga de datos;
- limpieza mínima y selección de variables;
- preparación diferenciada para variables numéricas y categóricas;
- entrenamiento de un clasificador;
- evaluación básica;
- persistencia del pipeline para reutilización.

2. Configuración del entorno

La siguiente celda instala dependencias solamente si faltan. Esto ayuda a que el notebook pueda ejecutarse en una computadora nueva sin pasos manuales previos.

```
import importlib.util
import subprocess
import sys

required_packages = {
    "pandas": "pandas",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "joblib": "joblib",
}

missing = [package for module, package in required_packages.items() if
            importlib.util.find_spec(module) is None]

if missing:
    print("Instalando dependencias faltantes:", missing)
    subprocess.check_call([sys.executable, "-m", "pip", "install", *missing])
else:
    print("Dependencias listas.")

from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn import set_config
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Configura scikit-learn para mostrar diagramas interactivos del pipeline en Jupyter.
set_config(display="diagram")

RANDOM_STATE = 42
```

3. Data Understanding

En esta etapa cargamos los datos, revisamos su estructura y buscamos señales iniciales de calidad como valores nulos, tipos de variables y distribución de la variable objetivo.

```
# Extracción de datos
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
data = pd.read_csv(url)

print(f"Filas: {data.shape[0]:,} | Columnas: {data.shape[1]:,}")
data.head()

# Vista rápida de tipos de datos y nulos
summary = pd.DataFrame({
    "tipo": data.dtypes.astype(str),
    "nulos": data.isna().sum(),
    "porcentaje_nulos": (data.isna().mean() * 100).round(2),
    "valores_unicos": data.nunique(),
})
summary.sort_values("porcentaje_nulos", ascending=False)

# Distribución de la variable objetivo
survival_distribution = (
    data["Survived"]
    .value_counts(normalize=True)
    .rename(index={0: "No sobrevivió", 1: "Sobrevivió"})
    .mul(100)
    .round(2)
    .rename("porcentaje")
)
survival_distribution
```

4. Filtrado y definición de variables

Eliminamos variables que no aportan valor predictivo generalizable en este ejemplo o que tienen demasiados valores nulos. Después separamos las variables independientes X de la variable objetivo y.

```
cols_to_drop = ["PassengerId", "Name", "Ticket", "Cabin"]

data_filtered = data.drop(columns=cols_to_drop)

X = data_filtered.drop(columns=["Survived"])
y = data_filtered["Survived"]

print("Columnas disponibles para el modelo:")
print(list(X.columns))

X.head()
```

5. Manejo de tipos de variables

Creamos dos pipelines internos:

- **Numérico:** imputación por mediana y escalado estándar.
- **Categorico:** imputación por moda y codificación One-Hot.

Luego unimos ambos flujos con ColumnTransformer.

```
numeric_features = ["Age", "SibSp", "Parch", "Fare"]
categorical_features = ["Pclass", "Sex", "Embarked"]

numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

# Compatibilidad entre versiones recientes y anteriores de scikit-learn.
try:
    one_hot_encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
except TypeError:
    one_hot_encoder = OneHotEncoder(handle_unknown="ignore", sparse=False)

categorical_transformer = Pipeline(steps=[
```

```

        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", one_hot_encoder),
    ])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features),
    ]
)

data_prep_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
])

data_prep_pipeline

```

6. Separación de entrenamiento y prueba

Separamos el dataset antes de ajustar transformadores para evitar fuga de información desde el conjunto de prueba.

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y,
)

print(f"Entrenamiento: {X_train.shape[0]:,} filas")
print(f"Prueba: {X_test.shape[0]:,} filas")

# Ajustamos el pipeline de preparación usando solo el conjunto de entrenamiento.
X_train_prepared = data_prep_pipeline.fit_transform(X_train)
X_test_prepared = data_prep_pipeline.transform(X_test)

print("Shape X_train preparado:", X_train_prepared.shape)
print("Shape X_test preparado:", X_test_prepared.shape)

feature_names = data_prep_pipeline.named_steps["preprocessor"].get_feature_names_out()

pd.DataFrame(X_train_prepared, columns=feature_names).head()

```

7. Diagrama del pipeline de preparación

Jupyter renderiza el objeto siguiente como un diagrama interactivo de scikit-learn.

```
data_prep_pipeline
```

8. Pipeline completo de modelado

Ahora agregamos un clasificador al preprocesador para tener un pipeline completo de Machine Learning. Esto permite llamar fit, predict y predict_proba sobre datos crudos sin repetir manualmente los pasos de preparación.

```

model_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1_000, random_state=RANDOM_STATE)),
])

model_pipeline.fit(X_train, y_train)
model_pipeline

y_pred = model_pipeline.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.3f}")
print("\nMatriz de confusión:")
print(confusion_matrix(y_test, y_pred))
print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred, target_names=["No sobrevivió", "Sobrevivió"]))

```

9. Prueba de inferencia

Usamos algunos registros del conjunto de prueba para confirmar que el pipeline recibe datos crudos, aplica las transformaciones necesarias y produce predicciones.

```
sample = X_test.head(5).copy()
sample_predictions = model_pipeline.predict(sample)
sample_probabilities = model_pipeline.predict_proba(sample)[: , 1]

resultados = sample.copy()
resultados["prediccion_survive"] = sample_predictions
resultados["probabilidad_survive"] = sample_probabilities.round(3)
resultados
```

10. Empaquetado y reutilización

Para compartir el pipeline entrenado, lo guardamos con `joblib`. Este archivo puede cargarse luego desde otro notebook, script o API.

```
models_dir = Path("models")
models_dir.mkdir(exist_ok=True)

model_path = models_dir / "titanic_survival_pipeline.joblib"
joblib.dump(model_pipeline, model_path)

print(f"Pipeline guardado en: {model_path.resolve()}")

loaded_pipeline = joblib.load(model_path)
loaded_pipeline.predict(sample)
```

11. Estructura sugerida para paquete Python con entry points

La forma moderna de empaquetar el pipeline es mover la lógica reutilizable a un paquete dentro de `src/` y declarar comandos en `pyproject.toml`. En este proyecto dejé una implementación lista en:

- `pipeline_entrypoints/pyproject.toml`
- `pipeline_entrypoints/src/titanic_entrypoints/pipeline.py`
- `pipeline_entrypoints/src/titanic_entrypoints/cli.py`
- `pipeline_entrypoints/Makefile`

La idea es que el notebook siga sirviendo para exploración, mientras que el paquete sirve para operación repetible desde terminal, CI/CD, Docker, cron, Airflow, GitHub Actions u otro orquestador.

12. Investigación: ¿qué es un entry point?

Según la especificación de PyPA, un entry point es un mecanismo para que una distribución instalada anuncie componentes que otros programas pueden descubrir y usar. El caso más común es `console_scripts`: al instalar el paquete, el instalador crea un comando de terminal que llama una función Python.

Un entry point tiene tres partes:

- **Grupo:** define el tipo de integración. Ejemplos: `console_scripts`, `gui_scripts` o un grupo propio para plugins.
- **Nombre:** identifica el comando o plugin dentro del grupo. En `console_scripts`, este será el comando disponible en la terminal.
- **Referencia al objeto:** apunta a un objeto importable, normalmente `paquete.modulo:funcion`.

Ejemplo conceptual:

```
[project.scripts]
titanic-train = "titanic_entrpoints.cli:train"
```

Después de instalar el paquete, `titanic-train` ejecuta la función `train()` del módulo `titanic_entrpoints.cli`.

13. Adaptación del pipeline de scikit-learn

El pipeline original del notebook ya hacía el flujo principal:

1. cargar datos de Titanic;
2. separar variables predictoras y objetivo;
3. imputar y escalar variables numéricas;
4. imputar y codificar variables categóricas;
5. entrenar `LogisticRegression`;
6. guardar el pipeline completo con `joblib`.

Para usar entry points se separó esa lógica en funciones dentro del paquete `titanic_entrpoints`. La adaptación importante es que el entrenamiento y la predicción ya no dependen de ejecutar celdas manualmente, sino de funciones invocables por comandos instalables.

Resumen de la adaptación creada en `pipeline_entrpoints/src/titanic_entrpoints/pipeline.py`

```
from pathlib import Path
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

MODEL_PATH = Path("models") / "titanic_survival_pipeline.joblib"

def build_pipeline(random_state=42) -> Pipeline:
    return Pipeline(steps=[
        ("preprocessor", build_preprocessor()),
        ("classifier", LogisticRegression(max_iter=1_000, random_state=random_state)),
    ])

def train_pipeline(source, model_path=MODEL_PATH, test_size=0.20):
    data = load_data(source)
    X, y = split_features_target(data)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )

    model = build_pipeline()
    model.fit(X_train, y_train)
    joblib.dump(model, model_path)
    return {"model_path": str(model_path)}

def predict_survival(input_csv, model_path=MODEL_PATH, output_csv=None):
    model = joblib.load(model_path)
    data = pd.read_csv(input_csv)
    result = data.copy()
    result["prediccion_survive"] = model.predict(data)
    result["probabilidad_survive"] = model.predict_proba(data)[:, 1].round(3)
    if output_csv:
        result.to_csv(output_csv, index=False)
    return result
```

La interfaz de línea de comandos quedó en `pipeline_entrpoints/src/titanic_entrpoints/cli.py`. Cada función `train`, `predict` y `main` usa `argparse` para recibir parámetros desde terminal y devuelve un código de salida entero, que es el patrón esperado por `console_scripts`.

```
# Ejemplo simplificado de funciones objetivo para los entry points.

import argparse
from pathlib import Path

def train(argv=None) -> int:
    parser = argparse.ArgumentParser(description="Entrena el pipeline Titanic.")
    parser.add_argument("--source", default=TITANIC_URL)
    parser.add_argument("--model-path", default=str(MODEL_PATH))
    args = parser.parse_args(argv)

    train_pipeline(source=args.source, model_path=Path(args.model_path))
    return 0

def predict(argv=None) -> int:
    parser = argparse.ArgumentParser(description="Predice supervivencia con el pipeline entrenado.")
    parser.add_argument("input_csv")
    parser.add_argument("--model-path", default=str(MODEL_PATH))
    parser.add_argument("--output-csv", default=None)
    args = parser.parse_args(argv)

    predict_survival(args.input_csv, args.model_path, args.output_csv)
    return 0
```

14. Declaración de entry points en pyproject.toml

La especificación de pyproject.toml indica que [project.scripts] corresponde al grupo console_scripts. La adaptación del paquete usa esa tabla para exponer tres comandos:

```
[project.scripts]
titanic-pipeline = "titanic_entrypoints.cli:main"
titanic-train = "titanic_entrypoints.cli:train"
titanic-predict = "titanic_entrypoints.cli:predict"
```

También se agregó un grupo propio para descubrimiento tipo plugin:

```
[project.entry-points."titanic_entrypoints.pipeline_builders"]
logistic-regression = "titanic_entrypoints.pipeline:build_pipeline"
```

Esto permite que otro programa descubra constructores de pipelines instalados sin acoplarse directamente al módulo.

```
# Descubrimiento de un entry point personalizado usando importlib.metadata.
# Este patrón se usa cuando una aplicación quiere cargar plugins instalados.
```

```
from importlib.metadata import entry_points

def load_pipeline_builder(name="logistic-regression"):
    candidates = entry_points(group="titanic_entrypoints.pipeline_builders")
    for entry_point in candidates:
        if entry_point.name == name:
            return entry_point.load()
    raise LookupError(f"No se encontro el builder {name!r}")

# builder = load_pipeline_builder()
# model_pipeline = builder()
```

15. Uso del paquete adaptado

Desde terminal, el flujo queda así:

```
cd pipeline_entrypoints
pip install -e .
titanic-train --model-path models/titanic_survival_pipeline.joblib
titanic-predict pasajeros.csv --model-path models/titanic_survival_pipeline.joblib
--output-csv predicciones.csv
```

Con uv se podría ejecutar de forma equivalente dentro del proyecto:

```
cd pipeline_entrypoints
uv run titanic-train --model-path models/titanic_survival_pipeline.joblib
```

Con Poetry, tras declarar los scripts en `pyproject.toml`, se instala el proyecto y se ejecuta dentro del ambiente virtual:

```
poetry install
poetry run titanic-train --model-path models/titanic_survival_pipeline.joblib
```

Con Conda, una receta `meta.yaml` también puede declarar Python entry points dentro de `build.entry_points`.

16. Relación con un Makefile

Un Makefile sirve para automatizar tareas repetibles mediante **targets**, dependencias, acciones y variables. En este caso, el Makefile no reemplaza al entry point; lo invoca.

Ejemplo incluido en `pipeline_entrypoints/Makefile`:

```
MODEL_PATH = models/titanic_survival_pipeline.joblib
INPUT_CSV ?= pasajeros.csv
OUTPUT_CSV ?= predicciones.csv

install:
    pip install -e .

train:
    titanic-train --model-path $(MODEL_PATH)

predict:
    titanic-predict $(INPUT_CSV) --model-path $(MODEL_PATH) --output-csv $(OUTPUT_CSV)
```

La relación práctica es:

- el **entry point** define el comando estable de la aplicación Python;
- el **Makefile** coordina tareas del proyecto usando esos comandos;
- en CI/CD, `make train`, `make test` o `make predict` pueden llamar comandos Python sin repetir detalles de rutas o módulos.

17. Respuestas solicitadas

¿Qué usos tendría para ambientes de producción un entry point de Python?

En producción, un entry point permite convertir funciones Python en comandos estables e instalables. Para un pipeline de machine learning esto sirve para:

- ejecutar entrenamiento batch con `titanic-train`;
- ejecutar inferencia batch con `titanic-predict`;
- llamar tareas desde Docker, cron, Airflow, GitHub Actions, Makefiles u otros orquestadores;
- publicar una interfaz de operación sin pedir que alguien recuerde `python ruta/al/modulo.py`;
- versionar el comando junto con el paquete;
- cargar plugins o variantes de modelos con `importlib.metadata.entry_points()`.

¿Se podría utilizar un entry point con un manejador de paquetes?

Sí. Los entry points son metadata del paquete, por lo que funcionan cuando el proyecto se instala como distribución Python. `pip`, `uv` y `Poetry` pueden leer `pyproject.toml` y crear comandos desde `[project.scripts]`. `Conda` también puede crear Python entry points desde una receta `meta.yaml` usando `build.entry_points`.

Ejemplos:

```
pip install -e .
titanic-train --help

uv run titanic-train --help

poetry install
poetry run titanic-train --help
```

¿Qué relación podría tener un entry point con un Makefile?

El entry point es el comando Python empaquetado; el Makefile es una capa de automatización del proyecto. En vez de escribir rutas largas o comandos frágiles, el Makefile puede llamar `titanic-train` o `titanic-predict`. Así se estandarizan tareas locales y de CI/CD:

```
make install
make train
make predict INPUT_CSV=pasajeros.csv OUTPUT_CSV=predicciones.csv
```

En resumen: el entry point ofrece una interfaz de ejecución; el Makefile organiza cuándo y con qué parámetros se ejecuta.

18. Fuentes consultadas

- Python Packaging User Guide, **Entry points specification**: <https://packaging.python.org/en/latest/specifications/entry-points/>
- Python Packaging User Guide, **pyproject.toml specification**: <https://packaging.python.org/en/latest/specifications/pyproject-toml/>
- DataCamp, **GitHub Actions and MakeFile: A Hands-on Introduction**: <https://www.datacamp.com/tutorial/makefile-github-actions-tutorial>
- Poetry docs, **Commands / run**: <https://python-poetry.org/docs/cli/>
- uv docs, **Configuring projects / Entry points**: <https://docs.astral.sh/uv/concepts/projects/config/>
- conda-build docs, **Defining metadata / Python entry points**: <https://docs.conda.io/projects/conda-build/en/3.21.x/resources/define-metadata.html>
- Repositorio de la actividad: <https://github.com/donmatthiuz/MLOPS/tree/actividad9>

19. Conclusión

El notebook conserva el flujo de CRISP-DM y el pipeline de scikit-learn, pero ahora muestra cómo llevarlo a un formato más cercano a producción. La lógica del modelo se separó en funciones reutilizables, se expuso mediante entry points en `pyproject.toml`, y se conectó con un Makefile para automatizar instalación, entrenamiento e inferencia.

Esta estructura facilita repetir el entrenamiento, integrar el pipeline en CI/CD y ejecutar predicciones sin depender de celdas manuales de Jupyter.